What is GitHub? A beginner's guide

Git, GitHub, repositories, branches, pull requests and a first workflow, explained step by step.

1. The short answer

Git is a version control tool that runs on your computer. It records every change to your code as a snapshot called a commit, so you can see history, go back and work in parallel.

GitHub is a website that hosts Git repositories online. It adds collaboration on top of Git: sharing code, reviewing changes, tracking issues and running automation.

In one line: **Git is the tool, GitHub is the place where your Git projects live and where teams work together.**

2. Key words

Term	Meaning
Repository (repo)	A project folder that Git tracks, including its full history.
Commit	A saved snapshot of your changes, with a message describing them.
Branch	A separate line of work. The default branch is usually **main**.
Clone	Copy a remote repository to your computer.
Push / Pull	Push sends your commits to GitHub. Pull brings GitHub's new commits to you.
Pull Request (PR)	A request to merge your branch into another branch, with review and discussion.
Fork	Your own copy of someone else's repository on GitHub, used to propose changes.
Issue	A tracked task, bug or question inside a repository.

3. Setting up

You need Git installed on your computer and a free GitHub account. Then tell Git who you are:

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
```

4. Your first repository

Option A: create on GitHub and clone. Click **New** on GitHub, name the repo, then copy its URL and run:

```
git clone https://github.com/your-name/your-repo.git
cd your-repo
```

Option B: start locally and push. Create the repo on GitHub without any files, then in your project folder:

```
git init
git add .
git commit -m "Initial commit"
git branch -M main
git remote add origin https://github.com/your-name/your-repo.git
git push -u origin main
```

5. The everyday loop

```
git status                  # what has changed?
git add .                   # stage all changes
git commit -m "Add login"   # save a snapshot
git push                    # send it to GitHub
git pull                    # get others' changes
```

6. Branches and pull requests

Never work directly on **main** when you are changing something that others depend on. Make a branch, work on it safely, then open a pull request to merge it back.

```
git checkout -b login-feature     # create and switch to a branch
# ...make changes, then commit...
git push origin login-feature      # publish the branch
# On GitHub: click 'Compare & pull request', review, then Merge.
```

Once merged, the team's **main** branch includes your work and the live version stays safe until then.

7. Useful GitHub features

- **README.md**: the front page of your repo. Explain what it does and how to run it.
- **.gitignore**: list files Git should not track, such as dependencies, secrets and build output.
- **Issues**: track bugs, tasks and feature ideas.
- **Actions**: run automated checks or deploys whenever you push or open a PR.
- **Releases**: publish tagged versions of your project for others to download.
- **Stars and Forks**: bookmark a repo and copy it to experiment.

8. Good habits

- Write small, focused commits with clear messages, such as "Fix login redirect", not "stuff".
- Pull before you start work, so your copy is up to date.
- Never commit passwords, API keys or tokens. Use environment variables and .gitignore.
- Ask for a review before merging, even on small projects.
- Keep main working. Use branches for anything risky.

Quick command cheat sheet

Command	What it does
git status	Show changed and staged files
git log --oneline	Show commit history, compactly
git checkout -b NAME	Create and switch to a new branch
git switch main	Switch back to main
git diff	Show unstaged changes
git stash	Set changes aside temporarily
git merge NAME	Merge another branch into the current one